Hitesh Rathore(HR) Notes

TYPE CONVERSION IN JAVASCRIPT

FIRST PRINCIPLES • VISUAL NOTES

WHAT IS TYPE CONVERSION?

Changing a value from one data type to another.

"10" → 10
String → Number

WHY DOES JAVASCRIPT NEED TYPE CONVERSION?

JS must decide how to treat values in operations.

"10" + 5 ??
15 or "105"

TWO TYPES OF CONVERSION

① Implicit (Automatic)
JS does it itself.
"5" * 2 → Number("5") → 5 → 5 * 2 → 10

② Explicit (Manual)
Developer does it.
Number("5") → 5

STORY TIME

Hitesh wants to add "5" + 2

1. Hitesh writes "5" + 2 → "5" + 2
2. JS thinks: One is string, one is number. What should I do?
3. For + operator, JS prefers string concatenation. → "5" + "2"
4. Converts 2 to "2" and joins. → "5" + "2" = "52"

Result → "52"

COMMON CONVERSIONS

String → Number

Number("123")	→ 123
Number("12.5")	→ 12.5
Number("abc")	→ NaN
Number("")	→ 0
Number(" ")	→ 0

Whitespace is removed first.

Number → String

String(100)	→ "100"
String(true)	→ "true"
String(null)	→ "null"
String(undefined)	→ "undefined"

Converts value to its text form.

String → Boolean

Boolean("")	→ false
Boolean("Hello")	→ true
Boolean("false")	→ true
Boolean("0")	→ true

Only empty string is false.

Number → Boolean

Boolean(0)	→ false
Boolean(-0)	→ false
Boolean(0n)	→ false
Boolean(NaN)	→ false
Boolean(100)	→ true
Boolean(-5)	→ true

0, -0, 0n, NaN are false.

Null Conversions

Number(null)	→ 0
String(null)	→ "null"
Boolean(null)	→ false

null is absence of any value.

Undefined Conversions

Number(undefined)	→ NaN
String(undefined)	→ "undefined"
Boolean(undefined)	→ false

undefined is uninitialized value.

TRUTHY & FALSY

Falsy Values (Only these) ☒	Everything else is Truthy ✓
false	"hello"
0	[]
-0	{}
0n	function(){}
"" (empty string)	Infinity
null	-1
undefined	100
NaN	"0"
	new Date()
All these are treated as false in boolean context.	All these are treated as true in boolean context.

Meaning
Falsy: Values that are treated as false in boolean context.
Truthy: Values that are treated as true in boolean context.
Example

```
if ("") {
  // Won't run
}
```

```
if ([]) {
  // Will run
}
```

NaN – FULL EXPLANATION

NaN = Not a Number

① What is NaN?
NaN is a special value used to represent an invalid number result.
Type: number
typeof NaN → "number"

② When is NaN produced?
- Number("abc")
- "abc" - 1
- 0 / 0
- Infinity - Infinity
- parseInt("abc")
- Math.sqrt(-1)

③ Important Facts
NaN is not equal to anything, even NaN itself.
NaN == NaN → false
NaN === NaN → false
- isNaN(NaN) → true
- Number.isNaN(NaN) → true

NaN means:
"This operation doesn't return a valid number."

KEY TAKEAWAYS

- ☑ JS converts values automatically (implicit).
- ☑ You can convert manually (explicit).
- ☑ Know falsy values – only 8 of them.
- ☑ Everything else is truthy.
- ☑ NaN means invalid number result.
- ☑ Truthy & Falsy used in conditions, loops, defaults, validations, and more.

Remember This Mantra
If it feels empty → Falsy
If it has value → Truthy

SPECIAL OPERATION CASES

①	②	③	④	⑤	⑥	⑦	⑧
"5" + 2	"5" - 2	"5" * "2"	"5" / "2"	true + 1	false + 1	null + 1	undefined + 1
"52"	3	10	2.5	2	1	1	NaN
+ prefers concatenation	- requires numbers	Both become numbers	Both become numbers	true → 1	false → 0	null → 0	undefined → NaN

DOUBLE EQUALS (==)
Allows conversion
5 == "5" → true
"5" becomes 5

TRIPLE EQUALS (===)
No conversion
5 === "5" → false
How === works
① Data Type (Strict Check)
② Value (If types are same)
Different types → false

NOTE: == allows type conversion before comparison. === does not allow conversion.

NOTE

Null (==) and Undefined (==) are equal to each other, but none of them is equal to any other value.

`null == undefined` → true

Neither null nor undefined is equal to any other value (number, string, boolean, object, etc.).

1 BASIC COMPARISONS

Expression	Explanation	Result
null == undefined	Null is loosely equal to undefined	true
null === undefined	Null is not strictly equal to undefined because data types are different	false
null == 0	Null is not equal to any number	false
null == ""	Null is not equal to any string	false
null == false	Null is not equal to any boolean	false
null == []	Null is not equal to any object/array	false
null == {}	Null is not equal to any object	false
undefined == 0	Undefined is not equal to any number	false
undefined == ""	Undefined is not equal to any string	false
undefined == false	Undefined is not equal to any boolean	false
undefined == []	Undefined is not equal to any object/array	false
undefined == {}	Undefined is not equal to any object	false

KEY POINT
Only these two are loosely equal (==):
null == undefined → true
Every other comparison with null or undefined is false.

2 WHY DOES null == undefined RETURN true?

- In JavaScript, == (loose equality) performs type conversion before comparison.
- Both null and undefined are considered "empty" or "absence of value".
- According to the ECMAScript specification, they are treated as equal in loose comparison.
- But === (strict equality) compares both value and type — and they are different types.

```
typeof null          // 'object' (quirk in JS)
typeof undefined     // 'undefined'

null == undefined    // true
null === undefined   // false
```

KEY TAKEAWAY
- null == undefined → true (loose equality)
- null === undefined → false (strict equality)
- This is a special rule in JavaScript for loose equality (==).

3 EXAMPLES WITH EXPLANATION

Expression	Step by Step Explanation	Result
null == undefined	• null and undefined have a special equality rule in JavaScript. • They are considered equal only to each other in loose equality (==).	true
null == 0	• JavaScript does NOT convert null to 0 in == comparison. • null is only loosely equal to undefined. • Therefore null == 0 is false.	false
undefined == ""	• undefined is not converted to an empty string. • They are different values. • Therefore they are not equal.	false
null == false	• JavaScript does NOT convert null to 0 in == comparison. • null is only loosely equal to undefined. • Therefore null == false is false.	false
undefined == false	• undefined is only loosely equal to null. • It is not equal to false. • Therefore the result is false.	false

KEY TAKEAWAY
- In loose equality (==), null is only equal to undefined.
- null is NOT equal to any other value (0, false, "", etc.).
- undefined is only equal to null, not to any other value.

4 PRACTICAL CHECK

```
console.log(null == undefined);   // true
console.log(null === undefined);  // false
console.log(null == 0);           // false
console.log(undefined == 0);      // false
console.log(null == "");          // false
console.log(undefined == "");     // false
```

EXPLANATION
- null == undefined is true because == performs type coercion and treats both as equal.
- === checks both type and value, so null and undefined are not strictly equal.
- null and undefined are not equal to any other value (number, string, etc.).

Key Takeaway
== : checks type coercion + value
=== : checks type and value both

1 NULL COMPARISON BEHAVIOR IN JAVASCRIPT

LOOSE EQUALITY (==) vs RELATIONAL COMPARISONS (<, >, <=, >=)

```
console.log(null == 0);    // false
console.log(null < 0);     // false
console.log(null > 0);     // false
console.log(null <= 0);    // true
console.log(null >= 0);    // true
```

LOOSE EQUALITY (==)
- null is only loosely equal to undefined.
- It is NOT equal to any other value.
- null is NOT converted to 0 in == comparison.

null == undefined → true
null == 0 → false

RELATIONAL COMPARISONS (<, >, <=, >=)
- For relational comparisons, null is converted to 0.
- Then normal numeric comparison happens.

null → 0

```
0 < 0   → false
0 > 0   → false
0 <= 0  → true
0 >= 0  → true
```

EXPLANATION
- In loose equality (==), null is only equal to undefined.
- In == comparison, null is NOT converted to 0.
- In relational comparisons (<, >, <=, >=), null is converted to 0.
- After conversion, normal numeric comparison is performed.
- Therefore: null < 0 and null > 0 are false.
- And null <= 0 and null >= 0 are true.

QUICK SUMMARY
Equality (==): null == undefined → true; null == anything else → false
Relational (<, >, <=, >=): null → 0, then compare with the number

2 UNDEFINED EQUALITY (==) CHECKS

```
console.log(undefined == 0);   // false
console.log(undefined < 0);    // false
console.log(undefined > 0);    // false
console.log(undefined <= 0);   // false
console.log(undefined >= 0);   // false
```

undefined is not converted to any number. So, every comparison results in false.

EXPLANATION
- undefined is not converted to any number.
- So, undefined compared with 0 will always be false.
- That is why all comparisons using == with undefined are false.

IMPORTANT POINTS
- JS converts values to make operations possible.
- + behaves differently from other operators.
- == allows conversion, === does not.
- Falsy values become false in conditions.
- You should know special coercion cases.

COMMON MISTAKES
- Using == instead of ===
- Assuming "0" is false (it's true)
- Forgetting NaN is not equal to itself
- Relying on implicit conversion too much
- Not remembering [] + {} cases

PRO TIPS
- ✓ Use === for safe comparison
- ✓ Convert values explicitly when possible
- ✓ Remember falsy list
- ✓ Practice special cases
- ✓ Understand ToPrimitive concept

REVISION SUMMARY
1. JS has Implicit & Explicit Conversion
2. Know type ↔ type conversion rules
3. Understand operator behavior
4. Learn equality comparison deeply
5. Master special coercion cases

MASTER TYPE CONVERSION →
WRITE BETTER,
BUG-FREE JAVASCRIPT!

> "When JavaScript converts types, it's not magic. It's logic — hidden, but predictable."

JAVASCRIPT TYPE CONVERSION & EQUALITY

MCQ QUESTIONS

01 `console.log(null == undefined);`
A true B false C NaN D Error

02 `console.log(null === undefined);`
A true B false C NaN D Error

03 `console.log(null >= 0);`
A true B false C NaN D Error

04 `console.log(null > 0);`
A true B false C NaN D Error

05 `console.log(null <= 0);`
A true B false C NaN D Error

06 `console.log(undefined >= 0);`
A true B false C NaN D Error

07 `console.log(undefined == 0);`
A true B false C NaN D Error

08 `console.log(Number(null));`
A 0 B null C NaN D undefined

09 `console.log(Number(undefined));`
A 0 B undefined C NaN D Error

10 `console.log(Boolean(null));`
A true B false C null D Error

11 `console.log(Boolean("false"));`
A true B false C NaN D Error

12 `console.log(Boolean("0"));`
A true B false C NaN D Error

13 `console.log(Boolean([]));`
A true B false C NaN D Error

14 `console.log(Boolean({}));`
A true B false C NaN D Error

15 `console.log("" == false);`
A true B false C NaN D Error

16 `console.log("0" == false);`
A true B false C Error D NaN

17 `console.log(0 == false);`
A true B false C Error D NaN

18 `console.log(0 === false);`
A true B false C Error D NaN

19 `console.log("5" - 2);`
A "52" B 52 C 3 D Error

20 `console.log("5" + 2);`
A 7 B "52" C 3 D Error

21 `console.log([] == false);`
A true B false C Error D NaN

22 `console.log([] == ![]);`
A true B false C Error D NaN

23 `console.log(NaN == NaN);`
A true B false C Error D NaN

24 `console.log(NaN === NaN);`
A true B false C Error D NaN

25 `console.log(Number(""));`
A NaN B 0 C "" D undefined

26 `console.log(Number(" "));`
A NaN B 0 C undefined D Error

27 `console.log([] + []);`
A [] B "" C 0 D Error

28 `console.log([] + {});`
A "[object Object]" B "{}" C Error D NaN

29 `console.log({} + []);`
A "[object Object]" B "{}" C Error D NaN

30 `console.log(true + true + false);`
A 3 B 2 C 1 D true

JAVASCRIPT TYPE CONVERSION & EQUALITY

ANSWERS WITH EXPLANATION (1 - 10)

01 `console.log(null == undefined);`
Answer: A true
Explanation
- In loose equality (==), null and undefined are considered equal to each other.
- Therefore → true.

02 `console.log(null === undefined);`
Answer: B false
Explanation
- Strict equality (===) checks both value and type.
- null (object) and undefined (undefined type) are different types.
- Therefore → false.

03 `console.log(null >= 0);`
Answer: A true
Explanation
- In relational comparisons, null is converted to 0.
- So, 0 >= 0 → true.

04 `console.log(null > 0);`
Answer: B false
Explanation
- null is converted to 0.
- So, 0 > 0 → false.

05 `console.log(null <= 0);`
Answer: A true
Explanation
- null is converted to 0.
- So, 0 <= 0 → true.

06 `console.log(undefined >= 0);`
Answer: B false
Explanation
- undefined is converted to NaN in numeric context.
- Any comparison with NaN (except !=) is false.
- Therefore → false.

07 `console.log(undefined == 0);`
Answer: B false
Explanation
- undefined is converted to NaN in numeric context.
- NaN == 0 → false.

08 `console.log(Number(null));`
Answer: A 0
Explanation
- Number(null) converts null to 0.

09 `console.log(Number(undefined));`
Answer: C NaN
Explanation
- Number(undefined) converts undefined to NaN.

10 `console.log(Boolean(null));`
Answer: B false
Explanation
- Boolean(null) → false because null is a falsy value.

KEY CONVERSIONS & RULES
NaN → Not equal to anything, even NaN (both == and ===)
Falsy Values: false, 0, -0, 0n, "", null, undefined, NaN

11 `console.log(Boolean("false"));`
Answer: A true
Explanation
- "false" is a non-empty string.
- All non-empty strings are truthy.
- Result → true

12 `console.log(Boolean("0"));`
Answer: A true
Explanation
- "0" is a non-empty string.
- All non-empty strings are truthy.
- Result → true

13 `console.log(Boolean([]));`
Answer: A true
Explanation
- [] is an array.
- Arrays are objects in JavaScript.
- All objects are truthy.
- Result → true

14 `console.log(Boolean({}));`
Answer: A true
Explanation
- {} is an object.
- All objects are truthy.
- Result → true

15 `console.log("" == false);`
Answer: A true
Explanation
- false is converted to 0.
- "" is converted to 0.
- 0 == 0
- Result → true

16 `console.log("0" == false);`
Answer: A true
Explanation
- false is converted to 0.
- "0" is converted to 0.
- 0 == 0
- Result → true

17 `console.log(0 == false);`
Answer: A true
Explanation
- false is converted to 0.
- 0 == 0
- Result → true

18 `console.log(0 === false);`
Answer: B false
Explanation
- === checks both type and value.
- 0 is a number.
- false is a boolean.
- Different types.
- Result → false

19 `console.log("5" - 2);`
Answer: C 3
Explanation
- The - operator requires numeric operands.
- "5" is converted to 5.
- 5 - 2
- Result → 3

20 `console.log("5" + 2);`
Answer: B "52"
Explanation
- When one operand is a string, + performs string concatenation.
- 2 becomes "2".
- "5" + "2"
- Result → "52"

21 `console.log([] == false);`
Answer: A true
Explanation
- [] → ""
- false → 0
- "" → 0
- 0 == 0
- Result → true

22 `console.log([] == ![]);`
Answer: A true
Explanation
- [] → ""
- ![] → false
- false → 0
- "" → 0
- 0 == 0
- Result → true

23 `console.log(NaN == NaN);`
Answer: B false
Explanation
- NaN is not equal to anything, including itself.
- NaN == NaN
- Result → false

24 `console.log(NaN === NaN);`
Answer: B false
Explanation
- Even strict equality does not consider NaN equal to itself.
- NaN === NaN
- Result → false

25 `console.log(Number(""));`
Answer: B 0
Explanation
- Number("") converts an empty string to the number 0.
- Result → 0

26 `console.log(Number(" "));`
Answer: B 0
Explanation
- Whitespace is trimmed first.
- Number(" ") becomes Number("").
- Number("") returns 0.
- Result → 0

27 `console.log([] + []);`
Answer: B ""
Explanation
- [] → ""
- [] → ""
- "" + ""
- Result → ""

28 `console.log([] + {});`
Answer: A "[object Object]"
Explanation
- [] → ""
- {} → "[object Object]"
- "" + "[object Object]"
- Result → "[object Object]"

29 `console.log({} + []);`
Answer: A "[object Object]"
Explanation
- {} → "[object Object]"
- [] → ""
- "[object Object]" + ""
- Result → "[object Object]"

30 `console.log(true + true + false);`
Answer: B 2
Explanation
- true → 1
- true → 1
- false → 0
- 1 + 1 + 0
- Result → 2

JAVASCRIPT'S INTERNAL TYPE CONVERSION ALGORITHMS

Until now we learned:
Number() String() Boolean()

But a very important question remains:

Question: How does JavaScript actually convert values?
Example: Boolean("Hello") → Output: true

Question: Why true? Why not false? Who decides this?

Answer: Inside the JavaScript Engine there are internal conversion rules.
These rules are called:
ToBoolean() ToNumber() ToString()
☒ You cannot directly call them.
These are internal engine algorithms.

Think:
You write: Boolean("Hello")
But internally engine does: ToBoolean("Hello")
and returns the result. ✓

PART 1 ToBoolean()

Question: What is ToBoolean()?
Answer: An internal algorithm that converts a value into true or false.

Let's start from first principles.

Question: How many possible Boolean values exist?
Answer: Only two.
true ✓ false ☒

Therefore: Every value in JavaScript must eventually become: true or false when used in conditions.

Example:

```
if(value)
{
  ...
}
```

Engine internally does:

```
if(ToBoolean(value))
{
  ...
}
```

Engine internally does:

```
if(ToBoolean(value))
{
  ...
}
```

ToBoolean()

① THE MOST IMPORTANT RULE
ALMOST EVERYTHING → TRUE ✓
ONLY 8 SPECIAL VALUES → FALSE ☒

② FALSY VALUES
JavaScript Has Only 8 Falsy Values

① false	② 0
③ -0	④ 0n
⑤ ""	⑥ null
⑦ undefined	⑧ NaN

EVERYTHING ELSE IS TRUTHY ✓

③ EXAMPLES

①	②	③	④	⑤
ToBoolean(false)	ToBoolean(0)	ToBoolean("")	ToBoolean("Hello")	ToBoolean("0")
false ☒	false ☒	false ☒	true ✓	true ✓

④ QUESTION
Why does "0" become true ?
"0" → Non-Empty String → Truthy → true ✓
Answer:
Because: "0" is NOT an empty string.

⑤ QUESTION
What about: ToBoolean([])
Answer: true ✓
Why?
Array exists.
It is not one of the 8 falsy values.
Therefore: Truthy → true ✓

⑥ QUESTION
What about: ToBoolean({})
Answer: true ✓
Why?
Object exists.
It is not one of the 8 falsy values.
Therefore: Truthy → true ✓

⑦ MEMORIZE THIS
FALSY: false 0 -0 0n "" null undefined NaN
EVERYTHING ELSE: TRUTHY ✓

PART 2 ToNumber()

Question: What is ToNumber()? | Answer: An internal algorithm that converts values into numbers.

STRING → NUMBER
Example: ToNumber("100") → 100
Example: ToNumber("25") → 25

QUESTION
What happens with: ToNumber("hello") ?
Answer: Engine cannot create a valid number.
Result: NaN

EMPTY STRING
Question: What happens here? ToNumber(" ")
Many beginners think: NaN Wrong.
Output: 0
Reason:
Empty string is treated as: 0

WHITESPACE STRING
ToNumber(" ")
Output: 0
Question: Why?
Answer: Because engine first removes spaces.
Result: ""
Then: 0

BOOLEAN → NUMBER
ToNumber(true) → 1
ToNumber(false) → 0

NULL → NUMBER
ToNumber(null) → 0

UNDEFINED → NUMBER
ToNumber(undefined) → NaN

SUMMARY

"100"	""	" "	true	false	null	undefined
100	0	0	1	0	0	NaN

PART 3 ToString()

Question: What is ToString()? | Answer: An internal algorithm that converts values into strings.

① NUMBER → STRING
ToString(100) → "100"

② BOOLEAN → STRING
ToString(true) → "true"
ToString(false) → "false"

③ NULL → STRING
ToString(null) → "null"

④ UNDEFINED → STRING
ToString(undefined) → "undefined"

THE + OPERATOR MYSTERY
Question: Why does: "10" + 5 become "105" ?
Step 1 Engine sees: "10" + 5 (String + Number)
Step 2 + can perform concatenation.
Step 3 Engine converts: 5 → ToString() → "5" (using ToString)
Step 4 Now: "10" + "5" → "105"

THE - OPERATOR
Question: Why does: "10" - 5 become 5 ?
Because: Subtraction only works with numbers.
Engine performs: "10" → ToNumber() → 10
Then: 10 - 5 → 5

THE COMPARISON PROBLEM
Question: Why is: "5" == 5 true ?
Step 1 Engine sees: "5" == 5 (String and Number)
Step 2 == allows type conversion.
Step 3 Engine performs: "5" → ToNumber() → 5
Step 4 Compare: 5 == 5 → true

THE STRICT COMPARISON
"5" === 5
No conversion.
String vs Number
Different types.
Result: false

Question 1

CODE

```
let a = 10;
let b = a;
b = 30;
console.log(b);
console.log(a);
```

OUTPUT

```
30
10
```

EXPLANATION
The value of a remains unchanged.
Changing b does not affect a.

VS

CODE

```
let obj1 = {
  id: 20,
  name: "Rohit"
};
let obj2 = obj1;
obj2.id = 30;
console.log(obj1);
console.log(obj2);
```

OUTPUT

```
{
  id: 30,
  name: "Rohit"
}
{
  id: 30,
  name: "Rohit"
}
```

EXPLANATION
The change made through obj2 is also reflected in obj1.

Now let's know the exact diffrence between **Primitive** and **Non-primitive** data type

Primitive V/s Non-Primitive

FIRST PRINCIPLES APPROACH

PART 1 PRIMITIVE VALUES
Primitive means: A value that represents itself directly.
Primitive values are immutable.
QUESTION: What does immutable mean? → ANSWER: Cannot be changed after creation.
EXAMPLE: `let name = "Hitesh";`
QUESTION: Can we change the first character? `name[0] = "R";`
RESULT: Nothing changes. name → "Hitesh"
WHY? Because strings are immutable.

IMPORTANT DIFFERENCE
Primitive ↓ Immutable
Non-Primitive ↓ Mutable

PART 2 NON-PRIMITIVE VALUES
Also called: Reference Types
Non-Primitive values are mutable.

```
let user = {
  name: "Hitesh",
  age: 20
}
```

`user.name = "Rahul";`
RESULT

```
{
  name: "Rahul",
  age: 20
}
```

WHY? Objects are mutable.

HEAP AND STACK

STACK
Stack is a memory region that stores primitives stack me store hote hai in a structured (LIFO) manner.

HOW IT WORKS
- Values are pushed onto the stack.
- Values are popped from the top.
- Each value gets its own space.

– TOP –
value 4
value 3
value 2
value 1
STACK GROWS DOWNWARDS

EXAMPLE

```
let a = 10;
let b = 20;
let c = 30;
let d = 40;
```

Stored in Stack (One by one)

FIRST THOUGHT PRINCIPLE
Think of a stack of plates. You add a plate on top. You remove the plate from top.

PRIMITIVE GETS STORED IN STACK
WHY?
- Primitive values are usually small and have fixed size.
- Fast to store.
- Fast to access.
- Stack uses contiguous memory and follows LIFO, so operations are very fast.

HEAP
Heap is a memory region that stores objects and complex data structures.
Non-Primitives gets stored in Heap

HOW IT WORKS
- Objects are created in heap.
- A reference (address) is stored in stack.
- Objects remain in heap until garbage collected.

STACK MEMORY	HEAP MEMORY
refA 0x101	Object A { name: "Hitesh" }
refB 0x205	Object B { name: "Rahul" }
refC 0x309	Object C [1, 2, 3]

FIRST THOUGHT PRINCIPLE
Think of a big storage room. You put items anywhere. You keep the location (address) in a note.

EXAMPLE

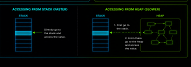
```
let user = { name: "Hitesh" };
let arr = [1, 2, 3];
let obj = { id: 1 };
```

References (addresses) are stored in Stack, actual objects are stored in Heap.

STACK IS FASTER IN ACCESS AND **HEAP** IS LITTLE SLOW THAN STACK

REASON: If we want to access something from the stack, we directly go to the stack and access it.

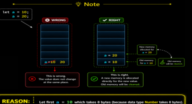
REASON: If we want to access something from the heap, we first go to the stack, and from there we go to the heap and then access it.



ACCESSING FROM STACK (FASTER)
STACK
Directly go to the stack and access the value.
ACCESSING FROM HEAP (SLOWER)
STACK
HEAP
1. First go to the stack.
2. From there go to the heap and access the value.

Note

```
let a = 10;
    a = 20;
```



This is wrong. The value does not change at the same place.

This is right. A new memory is allocated directly for the new value. Old memory will be cleaned.

REASON:
Let first a = 10 which takes 8 bytes (because data type Number takes 8 bytes).
Now we assign a string to a whose size is more than 8 bytes.
We cannot store this string at the previous memory location since it won't fit.
Even if the string size were less than 8 bytes, storing it at the same location would cause memory wastage.
Therefore, the rule is that on re-assignment, memory is reallocated.

Q1

```
let a = 10;
let b = a;
b = 20;
```

QUESTION: Will the value of a change?

SOLUTION: No.
Final Values: a → 10, b → 20
Reason: Primitive values are copied by value.
Initially: a 10, b 10 → b becomes 20 → a 10, b 20

Q2

```
let a = {
  name: "Hitesh"
};
let b = a;
```

QUESTION: When executing let b = a; Is the entire object copied?

SOLUTION: No. Only the reference is copied.
Stack: a, b → Heap: { name: "Hitesh" }

Q3

```
let a = {
  name: "Hitesh"
};
let b = a;
b.name = "Rahul";
```

QUESTION: What will be the value of a.name ?

SOLUTION: "Rahul"
Reason: Both variables point to the same object.
Heap: { name: "Rahul" }

Q4

```
let a = [1,2,3];
let b = a;
b.push(4);
```

QUESTION: What will be printed? console.log(a);

SOLUTION: [1,2,3,4]
Reason: Both variables reference the same array.
Heap: [1, 2, 3, 4]

Q5

```
let a = {
  name: "Hitesh"
};
let b = a;
b = {
  name: "Rahul"
};
```

QUESTION: Will a change?

SOLUTION: No.
Result: a.name → "Hitesh"
Reason: b now points to a different object.
Before: a, b → Object#101 { name: "Hitesh" }
After: a → Object#101 { name: "Hitesh" }; b → Object#102 { name: "Rahul" }

Q6

```
let a = {};
let b = {};
console.log(a == b);
```

QUESTION: What will be the output?

SOLUTION: false
Reason: Two different objects create two different references.
a → Ref#101, b → Ref#102
Ref#101 ≠ Ref#102

Q7

```
let a = {};
let b = a;
console.log(a == b);
```

QUESTION: What will be the output?

SOLUTION: true
Reason: Both variables contain the same reference.
a, b → Ref#101

Q1

Code

```
const limit = 100;
limit = 200;
console.log(limit);
```

Question: What will be the output of the above code?

Solution: Error (TypeError)

Why? You cannot reassign a value to a variable declared with const.

Explanation: When you declare a variable with const, you cannot reassign a new value to it. Trying to do so causes an error before execution.
Initial Declaration: Stack limit → 100
Reassignment (Not Allowed): Stack limit → ~~200~~
Memory View: Stack limit → 100 | Status: Reassignment blocked

Q2

Code

```
const obj1 = {
  name: "Hitesh",
  age: 20
}

obj1.age = 19;
// Is this valid?
```

Question: Is the above statement valid?

Explanation
obj1 is a constant reference that points to an object in memory.
When we change obj1.age, we are not creating a new object.
We are only updating the existing object's age property.

Initial State (Before obj1.age = 19)

Variable	Value (Address)
obj1	1000

points to → Heap (Memory for Objects): Address: 1000 { name: "Hitesh", age: 20 }

After Execution (After obj1.age = 19)

Variable	Value (Address)
obj1	1000

points to → Heap (Memory for Objects): Address: 1000 { name: "Hitesh", age: 19 }

Only the value of age changed. The address (1000) is still the same.

Analogy
obj1 is like the address of a house.
The address (1000) never changes.
We are just changing something inside the house (age), not the address of the house.

Important Note

- The "primitives are stored in stack and objects in heap" explanation is a learning model, not a strict ECMAScript rule.
- Different JavaScript engines may optimize storage differently.
- For learning and interviews, however, the Stack/Heap mental model above is exactly what you should use

★ Use the Stack/Heap model as a mental model for better understanding and in interviews.